pinelabs_fin — guide

Installation, the two main features, and how to call the API from your own buttons or other code.

---

1. How to install

What you need

Thing	Minimum version	Notes
Frappe	15.x	
ERPNext	15.x	Must be installed
Python	3.10 or newer	
MariaDB	10.6+ with InnoDB	Needed so the app can lock rows safely

> **Note:** To check your ERPNext version, run `bench --site <your-site> list-apps`.

Install steps

Open a terminal in your bench folder and run:

```
cd ~/frappe-bench
bench get-app https://github.com/<owner>/pinelabs_fin
bench --site <your-site> install-app pinelabs_fin
bench --site <your-site> migrate
```

The `migrate` step does three things:

- Makes the new tables (doctypes) the app needs: `Pinelabs Settings`, `Pinelabs Transaction`, and a few others.
- Adds 4 Modes of Payment: `PineLabs - Card`, `PineLabs - UPI`, `PineLabs - NetBanking`, `PineLabs - Wallet`.
- Adds 4 small fields on Mode of Payment under a "Pine Labs Integration" section.

> **Tip:** The app does **not** change any ERPNext built-in doctype. It does not add fields to Sales Invoice, POS Invoice, Payment Entry, Customer, or Contact. The only new fields are on Mode of Payment, which is just a setup doctype.

Turn on the scheduler

The scheduler runs a job every minute. This job is a safety net. If a webhook is lost or a cashier closes a window too soon, the scheduler still finishes the payment correctly.

```
bench --site <your-site> scheduler enable
bench --site <your-site> scheduler resume
```

Check it is on:

```
bench --site <your-site> scheduler status      # should say "Scheduler is enable
bench --site <your-site> show-pending-jobs     # should list 2 pinelabs_fin jobs
```

After install — check that everything works

- [ ] In ERPNext, **Pinelabs Settings** opens as a single page.
- [ ] **Mode of Payment** list shows the 4 new `PineLabs - *` rows.
- [ ] **Pinelabs Transaction** list opens (it will be empty for now).
- [ ] `bench scheduler status` says `enabled`.

If something is missing, run `bench migrate` again. It is safe to run it many times.

---

2. Feature 1 — Pay on Machine (terminal)

When to use it

The customer is **standing in front of you** and ready to tap a card or scan UPI on the Pine Labs machine.

How to use it

1. Open the submitted Sales Invoice (or POS Invoice / Payment Request) in ERPNext.
2. Click **Pay on Machine** at the top of the page.
3. A small window opens with a Mode of Payment list. Pick **PineLabs - Card** or **PineLabs - UPI**.
4. Click **Start Payment**.
5. A "Waiting for customer..." window appears. The Pine Labs machine beeps with the bill.
6. The customer taps their card or scans UPI.
7. In a few seconds, the window changes to "Status: SUCCESS" and closes on its own.
8. The invoice refreshes. Outstanding is now 0. A new Payment Entry shows under Payments.

Time taken: about 5 to 10 seconds. Most of that is the customer using the machine.

If the button is missing

The button is hidden when any of these is true:

- The invoice is not submitted (it is still a draft, or it was cancelled).
- Outstanding is already 0 (the invoice is paid).
- In Payable Documents, **Pay on Machine** is unticked or **Enabled** is unticked.

If you think the button should be there but it is not, open the browser developer tools (press F12) and look at the Console tab. Lines that start with `[Pinelabs] skip:` tell you why the button was hidden.

What to do if it fails

> **Safety: no double-charge.** Five layers of protection apply, including a row-level lock and **DB-level UNIQUE constraints** on `payment_id` and `order_id`. Even if a webhook retry, the per-minute scheduler, and a manual Refresh Status all race for the same transaction, only one Payment Entry is created. The app also blocks a second click while a payment is still INITIATED or PENDING — you will see: "A Terminal payment (PLT-...) is already in progress for this document." A fresh attempt is only allowed after the first reaches SUCCESS / FAILED / CANCELLED. Details: see "How no-duplicate is guaranteed" at the bottom of README.md.

Problem	What you see	What to do
Card was rejected	Red message: "Payment FAILED"	The payment is now FAILED (a final state). Click the button again to start a fresh attempt — the app allows this because the first attempt is no longer "in progress".
Customer waited too long	Window stays open	Click **Cancel** in the window. The payment is CANCELLED. The Pine Labs machine cancels too.
Cannot reach Pine Labs	Error: "Failed to reach Pine Labs"	Check your internet and try again.
You closed the window by mistake	Payment is still PENDING	**Do not click the button again** — the app will reject the second click with "A Terminal payment is already in progress for this document." The scheduler will finish the first attempt in a minute. To check sooner, open the Pinelabs Transaction record and click **Refresh Status**.

Picking the right machine (more than one machine)

If you set **Routing Mode** to **Mapping** and added rules, the app picks the right machine on its own. It looks at who you are, which warehouse the invoice uses, and so on. You do not have to choose.

3. Feature 2 — Send Payment Link (online)

When to use it

The customer is **not in front of you**. Maybe it is a phone order, a remote customer, or you need the money in advance. The customer will pay later, on their own phone, in their own time.

How to use it

1. Open the submitted invoice in ERPNext.
2. Make sure the customer has an **email** or a **mobile number** filled in. Without one of these, the link cannot be sent.
3. Click **Send Payment Link**. There is no extra window — every click makes one link that works for all payment methods.
4. A message shows the payment link URL.
5. **Pine Labs sends the link to the customer by SMS and email on its own.** You do not write or send any messages.
6. The customer opens the link, picks a payment method (card, UPI, net banking, or wallet) and pays.
7. When the customer pays, Pine Labs tells the app. The app makes the Payment Entry, and Outstanding becomes 0.

Default link life: 1 day. You can change this in Pinelabs Settings → Plural API.

How the app finds the customer's email and mobile

The app needs the customer's email and mobile to give to Plural. It tries to find them in this order:

1. **Per-doctype Customer Contact Field Mapping** (set on the Payable Documents row).
2. Common fields on the invoice itself (`contact_mobile` , `contact_email` , `mobile_no` , `email_id`).
3. The customer's main Contact record.

If it still cannot find them, the click fails with a clear message that tells you what it tried. Fix the mapping or fill in the customer's record.

The simplest mapping for Sales Invoice:

```
Mobile — Fetch From: Linked DocType
  Link Field:        contact_person
  Linked DocType:    Contact
  Field in Linked:   mobile_no

Email  — Fetch From: Linked DocType
  Link Field:        contact_person
  Linked DocType:    Contact
  Field in Linked:   email_id
```

Tracking the customer's payment

After you click Send, a Pinelabs Transaction record is made with status PENDING. You can:

- Open the **Pinelabs Transaction** list to see all open links.
- Open one record and click **Refresh Status** to ask Plural for the latest state right now.
- Or wait — every minute the scheduler checks all open links. When the customer pays, status becomes SUCCESS within a minute.

What to do if it fails

> **Safety: no double-link, no duplicate Payment Entry.** While a payment link is PENDING (customer has not paid yet), clicking Send Payment Link again is **rejected** with "A Plural payment (PLT-...) is already in progress for this document." Wait for the first link to either be paid (SUCCESS), expire (FAILED), or cancel it manually before sending a new one. Webhook redeliveries are deduplicated at the database via `Pinelabs Webhook Event.event_id` UNIQUE — a replayed webhook returns 200 OK and never re-runs the handler. Details: see "How no-duplicate is guaranteed" at the bottom of README.md.

Problem	What you see	What to do
Customer has no email or mobile	Error message with details	Add the email/mobile to the customer, or fix the Contact Field Mapping.
Customer never pays	Link stays PENDING until it expires	The scheduler marks it FAILED after it expires. Then click Send Payment Link again to make a new link. (Trying to click before the first link expires is rejected.)
Wrong Plural login details	Error: "You are not authenticated"	Re-check `plural_client_id` and `plural_client_secret`. Use **Test Plural Connection** to check.
Update from Pine Labs did not arrive	Payment stays PENDING for over a minute	The scheduler will finish it. Or open the record and click **Refresh Status**.

---

4. Using the API in your own buttons

The two buttons the app adds — **Pay on Machine** and **Send Payment Link** — are just calls to the app's API. If you want the same payment behaviour from your own custom button (or from any other code you write), call the same API. You get the same result: same payment, same Payment Entry, same protection against duplicates.

You do not need any new credentials, secrets, or set-up to use the API. It is already turned on after you install the app. The user calling the API must be logged in to ERPNext (just like any normal form action).

When you would do this

- You want a payment button on a custom doctype the app does not know about yet.
- You want the button to look different (different label, different position, different colour).
- You want some other action to trigger the payment automatically — for example, a server-side rule that sends a payment link when an invoice is submitted.
- An external system needs to ask ERPNext to take a payment.

Do I have to add the doctype to Pinelabs Settings?

Your source doctype is…	Add a row in Payable Documents?
Sales Invoice / POS Invoice / Payment Request	**No.** It just works.
Any custom doctype (e.g. Service Booking, Membership, Custom Order)	**Yes — required.** Without it, the payment goes through on the machine but the app cannot finish writing the result back. The error tells you exactly what to do.

For a custom doctype, add a row in **Pinelabs Settings → Payable Documents** with:

- DocType: your doctype name
- Enabled: ✅
- Amount Field: the field that holds the payable amount (e.g. `total_amount`)
- Create Payment Entry: ❌ (unticked)
- Status Field: a field on your doctype to update after a successful payment (e.g. `payment_status`)
- Paid Status Value: what to put in that field (e.g. `Paid`)

You can untick **Pay on Machine** and **Send Payment Link** on this row if you only want the API to work, not the default buttons.

The two main API endpoints

Endpoint 1 — Pay on Machine

Method name: `pinelabs_fin.api.terminal.initiate_terminal_payment`

What you send:

Field	Required	Example	What it means
`reference_doctype`	Yes	`"Sales Invoice"`	Which doctype the payment is for
`reference_name`	Yes	`"ACC-SINV-2026-00054"`	The specific record
`mode_of_payment`	Yes	`"PineLabs - Card"`	Card or UPI on the machine
`amount`	No	`250.00`	Leave empty to take the doc's outstanding amount

What you get back (on success):

```
{
  "success": true,
  "transaction_name": "PLT-2026-00123",
  "status": "PENDING",
  "response_code": 0,
  "message": "Approved"
}
```

The machine now beeps with the bill. The customer taps their card or scans UPI. Use `check_terminal_status` (below) to poll for completion.

Endpoint 2 — Send Payment Link

Method name: `pinelabs_fin.api.plural.initiate_payment_link`

What you send:

Field	Required	Example	What it means
`reference_doctype`	Yes	`"Sales Invoice"`	Which doctype the payment is for
`reference_name`	Yes	`"ACC-SINV-2026-00054"`	The specific record
`flow`	No	`"all_methods"`	`"all_methods"` (default), `"upi_only"`, or `"card_only"`
`expiry_minutes`	No	`60`	How long the link is valid (default: 1 day)

What you get back (on success):

```
{
  "success": true,
  "transaction_name": "PLT-2026-00125",
  "payment_link_url": "https://uat-checkout.plural.in/pay/xxxxxx",
  "expires_at": "2026-05-12T10:53:27+05:30"
}
```

Pine Labs sends the link to the customer by SMS and email automatically.

Endpoint 3 — Check the terminal status (polling)

Method name: `pinelabs_fin.api.terminal.check_terminal_status`

Used only with Pay on Machine, to ask "is the customer done yet?".

Field	Required	Example
`transaction_name`	Yes	`"PLT-2026-00123"` (from endpoint 1)

What you get back:

```
{
  "status": "PENDING",
  "payment_id": null,
  "payment_method": null,
  "error_message": null
}
```

`status` will be one of: `INITIATED`, `PENDING`, `SUCCESS`, `FAILED`, `CANCELLED`. Stop polling once you see one of the last three.

A custom button example you can copy

Drop this in your doctype's client script (your_doctype.js):

```
frappe.ui.form.on("Your Custom DocType", {
  refresh(frm) {
    if (frm.doc.docstatus !== 1) return;  // only on submitted docs
    if ((frm.doc.outstanding_amount || 0) <= 0) return;

    frm.add_custom_button(__("Take Payment"), () => {
      frappe.prompt(
        {
          fieldtype: "Link",
          fieldname: "mode",
          label: __("Mode of Payment"),
          options: "Mode of Payment",
          reqd: 1,
          get_query() {
            return { filters: { enabled: 1, pinelabs_enabled: 1 } };
          },
        },
        (v) => start_payment(frm, v.mode),
        __("Pay on Machine"),
        __("Start"),
      );
    });
  },
});

function start_payment(frm, mode_of_payment) {
  frappe.call({
    method: "pinelabs_fin.api.terminal.initiate_terminal_payment",
    args: {
      reference_doctype: frm.doctype,
      reference_name: frm.docname,
      mode_of_payment,
    },
    freeze: true,
    freeze_message: __("Sending to terminal..."),
    callback(r) {
      const m = r.message || {};
      if (!m.success) {
        frappe.msgprint({ title: __("Failed"), message: m.message, indicator: "r
        return;
      }
      poll(frm, m.transaction_name);
    },
  });
}

function poll(frm, txn) {
```

```
  const timer = setInterval(() => {
    frappe.call({
      method: "pinelabs_fin.api.terminal.check_terminal_status",
      args: { transaction_name: txn },
      callback(r) {
        const status = (r.message || {}).status;
        if (["SUCCESS", "FAILED", "CANCELLED"].includes(status)) {
          clearInterval(timer);
          frappe.show_alert({
            message: __("Payment {0}", [status]),
            indicator: status === "SUCCESS" ? "green" : "red",
          });
          frm.reload_doc();
        }
      },
    });
  }, 3000);
}
```

For Send Payment Link, swap the `method` to `pinelabs_fin.api.plural.initiate_payment_link` and remove the polling — Pine Labs sends the link to the customer on its own.

Common error messages from the API

Error you see	What it means	How to fix
`A Terminal payment (PLT-...) is already in progress for this document.`	There is an unfinished payment on this same doc	Wait for it to finish, or open the Pinelabs Transaction record and cancel it
`No outstanding amount on ...`	The doc has no `outstanding_amount`, `grand_total`, or `amount` field with a value > 0	Pass `amount` explicitly in your API call, or fill in the standard amount field on the source doc
`Mode of Payment X is not Pine Labs enabled.`	The mode you picked is not marked as PineLabs-enabled	Open the Mode of Payment record and tick the Pine Labs Integration fields
`No Pinelabs machine resolves for this transaction.`	The app could not find a terminal to use	Mark a machine as Is Default in Pinelabs Settings, or add a Machine Mapping rule
`Customer email or mobile is required to send a payment link.`	The app could not find a way to message the customer (Send Payment Link only)	Fill in the customer's email/mobile, or set up Customer Contact Field Mapping in Payable Documents
`PE creation supports Sales Invoice / POS Invoice / Payment Request only. Got X. Either add support or set Create Payment Entry to 0 on this doctype's Payable Doctypes row.`	The customer paid but the doctype is custom and there is no Payable Documents row telling the app how to finish	Add a Payable Documents row for this doctype with **Create Payment Entry** unticked and a status field configured (see "Do I have to add the doctype…" above)

Same safety as the buttons

The API is the same code path the buttons use, so all the safety nets you get from clicking are also there for API calls:

- A second API call while one payment is in progress is rejected.
- A duplicate webhook is rejected by the database.
- A retry of the cron job never makes a second Payment Entry.

- Status transitions are protected by a row-level lock.

So you cannot create a double-charge by calling the API in unusual patterns. The protection is built into the API itself, not into the buttons.